

Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos

Día 4: Auditoría de Datos (QA)

3.1.4 Checklist: Semana 1 - Día 4: Auditoría de Datos (QA)

En esta etapa, el objetivo es certificar la **veracidad lógica** del dato. No basta con que el dato esté cargado; debe ser coherente para que los modelos de Minería de Datos que construiremos más adelante no generen conclusiones erróneas.

Hito: Auditoría de Datos (QA) y Perfilamiento Estadístico

- [SI] **Python:** Script de **perfilamiento ejecutado** y **estadísticas revisadas**.

1. Fase Python: Perfilamiento Estadístico (VS Code)

- **1.1. Perfilamiento Estadístico:**
 - **Instrucción:** Abre **VS Code** y en tu script **data_profiling.py**, utiliza el método **.describe()** de **Pandas**. Este comando genera automáticamente la **media**, **desviación estándar**, **valores mínimos**, **máximos** y **cuartiles**.
 - **Acción:** Ejecuta **print(df[['age', 'monthly_income', 'total_working_years']].describe())**.

```
# -----  
-----  
# Título: Análisis de Calidad de Datos - IBM HR  
#  
# Objetivo: Realizar un perfilamiento estadístico y validar la  
# coherencia lógica  
# de los datos de empleados (años en la empresa vs años totales).  
#  
# Descripción: Este script extrae datos de una base de datos  
# PostgreSQL, calcula  
# estadísticas básicas (media, máximo, etc.) de campos clave y  
# busca incongruencias  
# en la experiencia laboral de los empleados.  
#  
# Archivo Python: day04_data_profiling_02.py  
#  
# Archivo PNG: day04_data_profiling_02.png  
# -----  
-----  
  
# Biblioteca para manipulación y análisis de datos  
import pandas as pd
```

```

# Módulo para conectar con la base de datos
from sqlalchemy import create_engine
# Módulo para leer variables de entorno del sistema
import os
# Carga variables desde un archivo .env
from dotenv import load_dotenv

# 1. Configuración del entorno y conexión
# Carga las variables de entorno (credenciales) desde un archivo
.env oculto
load_dotenv()

# Crea la conexión a la base de datos PostgreSQL usando
credenciales seguras
engine = create_engine(
    f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASS')}

f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB
_NAME')}"
)

# 2. Definición de la consulta SQL
# Define la consulta SQL para extraer toda la información de la
tabla de empleados
query = """
    SELECT *
    FROM
        employee_master_dirty_data
    """

# 3. Función principal de perfilamiento
def data_profiling():
    """Realiza la extracción, análisis estadístico y validación de
    reglas."""

    # Lee la tabla SQL y la carga en un DataFrame de Pandas
    df = pd.read_sql(query, con = engine)
    # Imprime estadísticas descriptivas de columnas clave: edad,
    ingresos y antigüedad
    print("\n---- 🔍 Perfilamiento Estadístico de IBM HR ----")
    print(df[['age', 'monthly_income',
'years_at_company']].describe())

    # --- Validación de Reglas de Negocio ---

```

```

# Busca empleados cuya antigüedad en la empresa sea mayor
que el total de años trabajados (ilógico)
errors = df[df['years_at_company'] > df['total_working_years']]

# Verifica si se encontraron errores
if not errors.empty:
    # Reporta el número de errores encontrados
    print(f"\n⚠ ¡Alerta! Se encontraron {len(errors)} empleados
con más años en la empresa que años trabajados en total.")
    # Muestra los detalles de los empleados con errores
    print(errors[['employee_number', 'years_at_company',
'total_working_years']])
else:
    # Informa que no se encontraron incongruencias
    print("\n✅ Coherencia temporal: Validada.")

# 4. Ejecución del script
if __name__ == "__main__":
    # Llama a la función principal
    data_profiling()

```

- **Validación:** Revisa que el **valor min** de **age** sea **18** y el **max** de **monthly_income** no supere los **20,000**.
 - **Resultado PNG:** day04_data_profiling_02.png

```

----  🔍 Perfilamiento Estadístico de IBM HR ----
      age  monthly_income  total_working_years
count  3000.000000      2.836000e+03      2911.000000
mean    33.949333      6.656402e+04      12.146341
std      8.413146      9.449614e+04      7.213763
min     18.000000     -1.456470e+05      0.000000
25%     28.000000      2.626800e+04      7.000000
50%     32.000000      4.751650e+04     10.000000
75%     38.000000      9.095725e+04     16.000000
max     60.000000      2.536740e+06     41.000000

⚠️¡ALERTA! Se encontraron 17 empleados con más años en la empresa que años trabajados en total.
  employee_number  years_at_company  total_working_years
47                48                47                8.0
150               151                50               20.0
416               417                 2                1.0
1117              1118                44                9.0
1383              1384                46                5.0
1437              1438                49               21.0
1489              1490                36                8.0
1654              1655                36                8.0
1678              1679                40               12.0
1699              1700                41               13.0
1972              1973                35                7.0
2265              2266                35                7.0
2491              2492                36                8.0
2642              2643                42               14.0
2700              2701                35                7.0
2797              2798                36                8.0
2963              2964                51               23.0
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> 

```

- 1.2. Detección de Anomalías (Outliers):

- **Instrucción:** Identifica registros que están **fuera del comportamiento normal** (pero que son técnicamente posibles).
- **Acción:** Filtra empleados con **suelos extremadamente bajos**: `df[df['monthly_income'] < 1500]`.
- **Opción #1:**

```

# -----
#
# Título: Detección de Valores Atípicos en Recursos Humanos
#
# Objetivo: Identificar empleados con ingresos mensuales
#           inusualmente bajos.
#

```

```
# Descripción: Este script carga un dataset de RRHH, estandariza
los nombres de las
# columnas a 'snake_case' y filtra registros donde el
'monthly_income' es menor a 1500,
# para detectar posibles errores de entrada de datos o casos
especiales.
#
# Archivo Python: day04_analyze_data_low_income_01.py
#
# Archivo PNG: day04_analyze_data_low_income_01.png
# -----
-----
```

```
# Librería para manipulación y análisis de datos
import pandas as pd
# Módulo para leer/escribir archivos CSV
import csv
# Módulo para interactuar con el sistema operativo (rutas de
archivos)
import os
# Módulo para manejo moderno y multiplataforma de rutas
from pathlib import Path
# Módulo para expresiones regulares (búsqueda de patrones de
texto)
import re
```

```
# --- Configuración de Rutas ---
# Define la carpeta raíz del proyecto subiendo 3 niveles desde
donde está este script
base_dir = Path(__file__).resolve().parents[3]
# Construye la ruta exacta hacia la carpeta de datos crudos de
rrhh
target_raw_dir = base_dir / '02_data' / 'raw' / 'rrhh'
# Define el nombre del archivo y construye la ruta completa
rrhh_file_name = 'IBM_HR_Dirty_Practices'
rrhh_path_csv = os.path.join(target_raw_dir,
f"{rrhh_file_name}.csv")
```

```
# --- Funciones ---
def to_snake_case(name):
    """
    Convierte textos de CamelCase o PascalCase a snake_case (ej:
'MonthlyIncome' -> 'monthly_income').
    Mejora la legibilidad y estándar de las columnas.
    """
```

```

# Inserta guion bajo antes de letras mayúsculas seguidas de
minúsculas
s1 = re.sub('([A-Z][a-z]+)', r'\1_2', name)
# Convierte a minúsculas y separa con guion bajo
return re.sub('([a-z0-9])([A-Z])', r'\1_2', s1).lower()

# --- Proceso Principal ---
try:
    # Intenta leer el archivo CSV y cargarlo en un DataFrame de
    pandas
    df = pd.read_csv(rrhh_path_csv)

    # Aplica la función para estandarizar todos los nombres de
    columnas a snake_case
    # Nota: Se corrigió 'df.columns' a 'df.columns' para evitar error
    df.columns = [to_snake_case(col) for col in df.columns]

except Exception as e:
    # Si hay error (archivo no existe, error de lectura), avisa y deja el
    df como None
    print(f" ✖ Error al cargar o procesar el dataset: {e}")
    df = None

# --- Análisis de Datos ---
if df is not None:
    # Filtra el DataFrame buscando empleados con un ingreso
    menor a 1500
    monthly_income_outliers = df[(df['monthly_income'] < 1500)]

    # Imprime los resultados obtenidos
    print("\n---- Registros Fuera de Rango (Monthly Income) ----")
    print(f"Total registros con tarifa atípica:
    {len(monthly_income_outliers)}")
    print(f"\n{monthly_income_outliers[['employee_number',
    'monthly_income']]}")
    print("\n")

```

- Resultado PNG: day04_analyze_data_low_income_01.png

```

---- Registros Fuera de Rango (Monthly Income) ----
Total registros con tarifa atípica: 18

    employee_number  monthly_income
100              101        -26631.0
282              283        -14586.0
327              328        -38346.0
377              378           1.0
607              608           1.0
904              905           1.0
1287             1288        -37224.0
1398             1399        -42999.0
1460             1461           1.0
1479             1480           1.0
1722             1723           1.0
1887             1888           1.0
2164             2165        -94230.0
2305             2306          -1.0
2327             2328           1.0
2509             2510           1.0
2677             2678        -55583.0
2834             2835       -145647.0

```

- Opción #2:

```

# -----
#
# Título: Identificador de Empleados con Bajos Ingresos
#
# Objetivo: Detectar y listar empleados cuyo ingreso mensual es
# atípicamente bajo.
#
# Descripción: Este script se conecta a una base de datos
# PostgreSQL, extrae información
# de empleados y filtra aquellos con un 'monthly_income' menor a
# 1500 para
# su revisión.
#
# Archivo Python: day04_analyze_data_low_income_02.py
#
# Archivo PNG: day04_analyze_data_low_income_02.png
# -----
#
# Importa la librería para manejo de datos (DataFrames)
import pandas as pd
# Para gestionar la conexión a la BD
from sqlalchemy import create_engine

```

```

# Para leer variables de entorno del sistema
import os
# Para cargar credenciales desde un archivo .env
from dotenv import load_dotenv
# 1. Configuración de conexión
load_dotenv() # Carga las credenciales (host, usuario, pass)
ocultas en el archivo .env
# Crea el motor de conexión a PostgreSQL usando las variables de
entorno
engine = create_engine(
    f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASS')}

f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB
_NAME')}"
)
# 2. Definición de la consulta SQL
# Selecciona toda la información de la tabla maestra de
empleados
query = """
    SELECT *
    FROM
        employee_master__dirty_data
"""
# 3. Función principal para extraer y analizar datos
def data_low_income():
    try:
        # Intenta leer la base de datos y cargarla en un DataFrame de
Pandas
        df = pd.read_sql(query, con=engine)
    except Exception as e:
        # Si hay error (conexión, permisos), lo muestra y detiene la
función
        print(f"✗ Error al conectarse al dataset: {e}")
        return

    # --- PROCESAMIENTO DE DATOS ---
    # Filtra el DataFrame original: mantiene solo filas con
'monthly_income' menor a 1500
    monthly_income_outliers = df[(df['monthly_income'] < 1500)]
    # --- SALIDA DE RESULTADOS ---
    print("\n---- Registros Fuera de Rango (Monthly Income) ----")
    # Cuenta cuántos empleados cumplen con el criterio de bajo
ingreso
    print(f"Total registros con tarifa atípica:
{len(monthly_income_outliers)}")

```

```

# Muestra en pantalla solo el número de empleado y su sueldo
atípico
print(f"\n{monthly_income_outliers[['employee_number',
'monthly_income']]}")
print("\n")
# 4. Punto de entrada del script
if __name__ == "__main__":
    # Ejecuta la función principal
    data_low_income()

```

- Resultado PNG: day04_analyze_data_low_income_02.png

```

---- Registros Fuera de Rango (Monthly Income) ----
Total registros con tarifa atípica: 18

    employee_number  monthly_income
100                101      -26631.0
282                283      -14586.0
327                328      -38346.0
377                378         1.0
607                608         1.0
904                905         1.0
1287               1288      -37224.0
1398               1399      -42999.0
1460               1461         1.0
1479               1480         1.0
1722               1723         1.0
1887               1888         1.0
2164               2165      -94230.0
2305               2306        -1.0
2327               2328         1.0
2509               2510         1.0
2677               2678      -55583.0
2834               2835     -145647.0

```

- 1.3. Reporte de Consola:
 - Instrucción: El script debe imprimir un resumen de "Filas vs Columnas" y un conteo de valores nulos por columna usando `df.isnull().sum()`.
 - Validación: La terminal debe mostrar 0 nulos en todas las columnas críticas.
 - Opción #1:

```
# -----  
-----  
# Título: Limpieza y Análisis Inicial de Datos de RRHH  
#  
# Objetivo: Cargar un dataset de rotación de personal (HR Attrition)  
y estandarizar los  
# nombres de las columnas a formato 'snake_case' para facilitar  
su manejo.  
#  
# Descripción: El script localiza un archivo CSV específico en el  
proyecto, carga los  
# datos, aplica una transformación a los nombres de las columnas  
para que sean más legibles  
# (ej: "NombreColumna" -> "nombre_columna") y muestra un  
reporte de valores nulos.  
#  
# Archivo Python: day04_empl_null_validation_01.py  
#  
# Archivo PNG: day04_empl_null_validation_01.png  
# -----  
-----
```

```
# Importa la librería para manipulación de datos (DataFrames)  
import pandas as pd  
# Importa el módulo para trabajar con archivos CSV  
import csv  
# Importa funciones para interactuar con el sistema operativo  
import os  
# Importa Path para manejo de rutas de archivos más limpio  
from pathlib import Path  
# Importa el módulo de expresiones regulares para manipulación  
de texto  
import re
```

```
# --- Configuración de Rutas ---  
# Define la ruta base buscando 3 niveles arriba del archivo actual  
(para encontrar la raíz del proyecto)  
base_dir = Path(__file__).resolve().parents[3]  
# Define la carpeta donde se encuentra el archivo "raw" (crudo) de  
recursos humanos  
target_raw_dir = base_dir / '02_data' / 'raw' / 'rrhh'  
# Define el nombre del archivo sin extensión  
rrhh_file_name = 'IBM_HR_Dirty_Practices'  
# Une la ruta de la carpeta y el nombre del archivo para obtener la  
ruta completa del CSV
```

```

rrhh_path_csv = os.path.join(target_raw_dir,
f"{rrhh_file_name}.csv")

# --- Función para estandarizar nombres ---
def to_snake_case(name):
    """Convierte cadenas CamelCase o PascalCase a snake_case
(minúsculas y guiones bajos)."""
    # Inserta guion bajo entre minúsculas y mayúsculas
    s1 = re.sub('([A-Z][a-z]+)', r'\1_\2', name)
    # Convierte a minúsculas y separa con guion bajo las
mayúsculas restantes
    return re.sub('([a-z0-9])([A-Z])', r'\1_\2', s1).lower()

# --- Bloque Principal: Carga y Procesamiento ---
try:
    # Intenta cargar el archivo CSV en un DataFrame de pandas
    df = pd.read_csv(rrhh_path_csv)
    # Renombra las columnas aplicando la función snake_case a
cada una
    df.columns = [to_snake_case(col) for col in df.columns]
    print("\n✅ Datos cargados y columnas estandarizadas
correctamente.")
except Exception as e:
    # Si ocurre un error (archivo no encontrado, formato inválido),
muestra el error
    print(f"❌ Error al cargar o procesar el dataset: {e}")
    # Inicializa el DataFrame como None para indicar que no se
cargaron datos
    df = None

# --- Reporte de Calidad de Datos ---
# Si la carga fue exitosa, muestra información útil
if df is not None:
    print("\n--- Reporte de Valores Nulos por Columna ---")
    # Calcula y muestra la suma de valores nulos (vacíos) por
columna
    # Nota: Se debe usar df.isnull().sum() (con paréntesis)
    print(df.isnull().sum())
    print("-----")
    print("\n")

```

- Resultado PNG: day04_empl_null_validation_01.png

✓ Datos cargados y columnas estandarizadas correctamente.

--- Reporte de Valores Nulos por Columna ---

employee_id	0
age	0
attrition	0
business_travel	0
country	0
city	0
daily_rate	0
department	0
distance_from_home	0
education	0
education_field	0
employee_count	0
employee_number	0
environment_satisfaction	0
gender	0
hire_date	0
hourly_rate	0
job_involvement	0
job_level	0
job_role	0
job_satisfaction	0
marital_status	0
monthly_income	164
monthly_rate	0
num_companies_worked	0
over_18	0
over_time	0
percent_salary_hike	0
performance_rating	0
relationship_satisfaction	0
standard_hours	0
stock_option_level	0
total_working_years	89
training_times_last_year	0
work_life_balance	0
years_at_company	0
years_in_current_role	0
years_since_last_promotion	0
years_with_curr_manager	0

dtype: int64

- Opción #2:

```

#
=====
# Título: Validación de Datos Nulos en Base de Datos
#
# Objetivo: Conectarse a una base de datos PostgreSQL, extraer la
# tabla de
# empleados y mostrar un conteo de valores nulos (vacíos) por
# columna.
#
# Descripción: Este script automatiza la revisión de calidad de
# datos para
# identificar campos faltantes en 'employee_master_data'
# utilizando Pandas y
# SQLAlchemy.
#
# Archivo Python: day04_empl_null_validation_02.py
#
# Archivo PNG: day04_empl_null_validation_02.png
#
=====
=====

# --- IMPORTACIÓN DE LIBRERÍAS ---
import pandas as pd          # Librería para manipulación y análisis
                              # de datos
from sqlalchemy import create_engine # Motor para conectar
Python con SQL
import os                    # Para interactuar con el sistema operativo
                              # (leer variables de entorno)
from dotenv import load_dotenv # Para cargar credenciales
                              # secretas desde un archivo .env
# --- CONFIGURACIÓN ---
# Carga las variables de entorno (usuario, pass, host, etc.) del
# archivo .env
load_dotenv()
# Crea la conexión a la base de datos PostgreSQL usando los
# datos seguros
engine = create_engine(
    f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASS')}"

    f"@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB'
_NAME')}"
)

```

```

# --- FUNCIÓN PRINCIPAL ---
def empl_null_validation():
    """
    Realiza la consulta, carga los datos y cuenta los valores nulos.
    """

    # Define la consulta SQL para extraer todos los datos de la tabla
    de empleados
    query = """
        SELECT *
        FROM
            employee_master_dity_data
    """

    # Ejecuta la consulta y carga el resultado en un DataFrame de
    Pandas
    df = pd.read_sql(query, con = engine)
    # Imprime en consola la suma de valores nulos (NaN/None) por
    columna
    print("--- Reporte de Valores Nulos por Columna ---")
    print(df.isnull().sum())
    print("-----")

# --- PUNTO DE ENTRADA ---
if __name__ == "__main__":
    # Llama a la función si el script se ejecuta directamente
    empl_null_validation()

```

- **Resultado PNG: day04_empl_null_validation_02.png**

✓ Datos cargados y columnas estandarizadas correctamente.

--- Reporte de Valores Nulos por Columna ---

employee_id	0
age	0
attrition	0
business_travel	0
country	0
city	0
daily_rate	0
department	0
distance_from_home	0
education	0
education_field	0
employee_count	0
employee_number	0
environment_satisfaction	0
gender	0
hire_date	0
hourly_rate	0
job_involvement	0
job_level	0
job_role	0
job_satisfaction	0
marital_status	0
monthly_income	164
monthly_rate	0
num_companies_worked	0
over_18	0
over_time	0
percent_salary_hike	0
performance_rating	0
relationship_satisfaction	0
standard_hours	0
stock_option_level	0
total_working_years	89
training_times_last_year	0
work_life_balance	0
years_at_company	0
years_in_current_role	0
years_since_last_promotion	0
years_with_curr_manager	0

dtype: int64

- [Si] SQL: Ejecutadas las 2 consultas de validación cruzada (Temporalidad y Edad).

2. Fase SQL: Validación Cruzada y Coherencia (DBeaver)

- 2.1. Validación de Temporalidad:

- **Instrucción:** Ejecuta una consulta para encontrar empleados cuyos años en la empresa superen sus años totales de experiencia laboral.
- **Consulta:**

```
/*****  
*****/
```

Título: Detección de Inconsistencias en Antigüedad Laboral

Objetivo: Identificar registros de empleados con datos erróneos.

Descripción: Selecciona empleados cuya antigüedad en la empresa actual es mayor que sus años totales de experiencia laboral, lo cual es lógicamente incorrecto (un empleado no puede llevar más años en la empresa que el total de años que ha trabajado en su vida).

Archivo SQL: day04_audit_seniority.sql

Archivo CSV: day04_audit_seniority.csv

Archivo PNG: day04_audit_seniority.png

```
*****/
```

-- Seleccionamos los campos clave para identificar al empleado y revisar sus datos

SELECT

```
    employee_number,  -- Identificador único del empleado  
    years_at_company, -- Años en la empresa actual  
    total_working_years-- Años totales de experiencia laboral
```

FROM

```
    employee_master_dirty_data-- Tabla maestra de datos de  
    empleados
```

WHERE

```
-- Filtramos inconsistencias: Años empresa > Años totales  
    years_at_company > total_working_years;
```

- **Resultado PNG: day04_audit_seniority.png**

	employee_number	years_at_company	total_working_years
1	48	47	8
2	151	50	20
3	417	2	1
4	1,118	44	9
5	1,384	46	5
6	1,438	49	21
7	1,490	36	8
8	1,655	36	8
9	1,679	40	12
10	1,700	41	13
11	1,973	35	7
12	2,266	35	7
13	2,492	36	8
14	2,643	42	14
15	2,701	35	7
16	2,798	36	8
17	9,162	41	19

- **Validación:** El resultado esperado es **0 filas**. Si aparece alguien, es un error de lógica de origen.
- **2.2. Validación de Edad de Contratación:**
 - **Instrucción:** Calcula la edad a la que fue contratado el empleado para asegurar que cumple con las políticas de trabajo (mínimo 18 años).
 - **Consulta:**

```

/*****
*****

```

Título: Análisis de Contrataciones Menores de Edad

Objetivo: Identificar empleados contratados antes de cumplir los 18 años.

Descripción: Esta consulta selecciona datos básicos de empleados y calcula su edad de contratación para detectar posibles inconsistencias o incumplimientos de políticas de edad laboral.

Archivo SQL: day04_analyze_underage_hires.sql

Archivo CSV: day04_analyze_underage_hires.csv

Archivo PNG: day04_analyze_underage_hires.png

```

*****
*****/

```

SELECT

FROM

-- Filtro: Muestra solo casos donde la edad de contratación calculada es menor a 18

```
(age - years_at_company) < 18;
```

Archivo SQL: empl_promo_consistency_audit.sql

Archivo CSV: empl_promo_consistency_audit.csv

Archivo PNG: empl_promo_consistency_audit.png

```
*****  
*****/
```

-- Seleccionamos los datos necesarios para identificar al empleado y revisar sus tiempos

SELECT

employee_number, -- Número único de identificación del empleado
years_at_company, -- Años totales que el empleado lleva en la empresa
years_since_last_promotion -- Años transcurridos desde su última promoción registrada

FROM

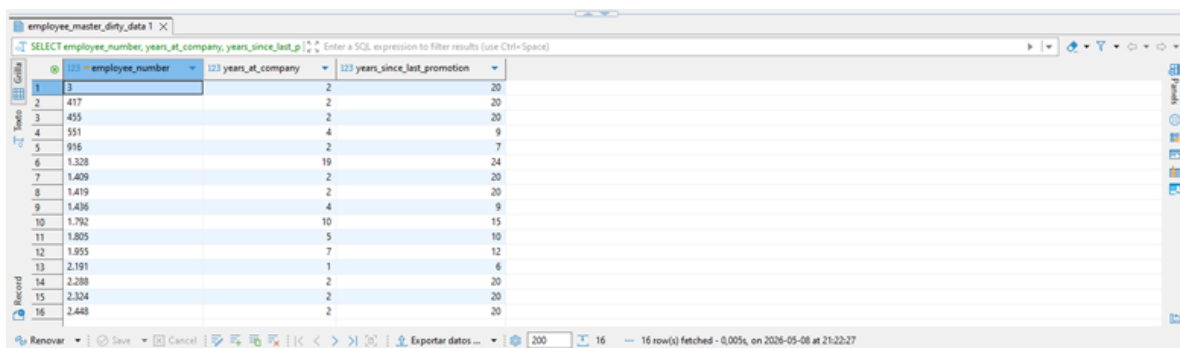
employee_master_dirty_data -- Tabla maestra que contiene la información laboral

WHERE

-- Filtramos para encontrar solo los registros inconsistentes:
-- donde la promoción ocurrió antes de que el empleado entrara a la empresa.

years_since_last_promotion > years_at_company;

- **Resultado PNG: empl_promo_consistency_audit.png**



	employee_number	years_at_company	years_since_last_promotion
1	3	2	20
2	417	2	20
3	455	2	20
4	551	4	9
5	916	2	7
6	1.328	19	24
7	1.409	2	20
8	1.419	2	20
9	1.436	4	9
10	1.792	10	15
11	1.805	5	10
12	1.955	7	12
13	2.191	1	6
14	2.288	2	20
15	2.324	2	20
16	2.448	2	20

- [Si] **PBI: Creado el medidor de salud del dato en el dashboard.**

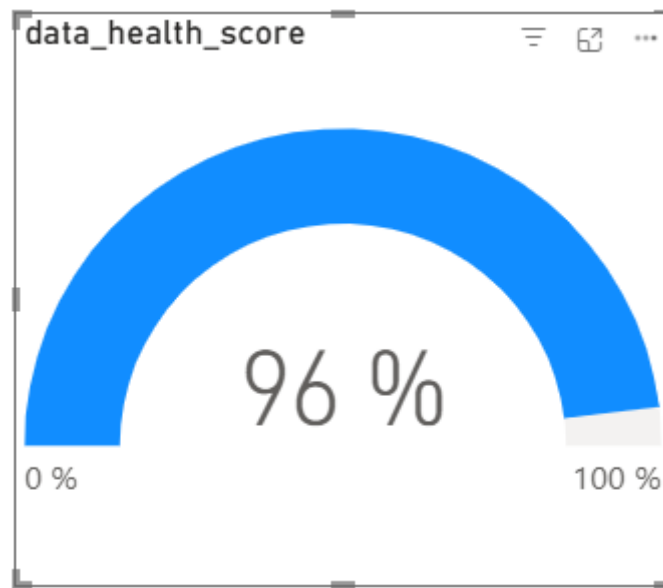
3. **Fase Power BI: Dashboard de Salud del Dato**

- **3.1. Medidor de Salud (Gauge):**
 - **Instrucción:** Crea la medida DAX Data_Health_Score.

- **Acción:** En la pestaña "Modelado", haz clic en **Nueva medida** y pega:

```
data_health_score = 1 - ('public  
employee_master_dirty_data'[total_logical_errors] / COUNT('public  
employee_master_dirty_data'[employee_number]))
```

- **Visual:** Arrastra esta medida a un objeto visual de **Medidor**.
- **Resultado PNG:** day04_data_health_score.png



- **3.2. Matriz de Hallazgos:**
 - **Instrucción:** Crea una tabla que liste los employee_number que fallaron en los checks de SQL.
 - **Filtro:** En el panel de filtros del objeto visual, arrastra Total_Errores y selecciona "Es mayor que 0".
 - **Resultado PNG:** day04_data_health_filter.png

employee_number	Suma de age	Suma de years_at_company	Suma de total_working_years	Suma de hire_age
11	35	5		30
31	33	1		32
48	37	47	8	-10
56	33	15		18
66	55	5		50
151	40	50	20	-10
180	38	2		36
188	55	24		31
196	37	8		29
199	38	1		37
247	33	2		31
250	45	3		42
Total	3494	1315	179	2179

- **3.3. Formato Condicional:**

- **Instrucción:**

- **Paso 4.1:** Selecciona la **tabla** creada en el Paso 3.3.
 - **Paso 4.2:** En el panel de **Formato**, busca **Elementos de celda**.
 - **Paso 4.3:** Activa el **Icono** para la columna **Hire_Age**.
 - **Paso 4.4:** Configura las reglas:
 - Si el valor es **< 18**, mostrar icono de **X Roja**.
 - Si el valor es **>= 18**, mostrar icono de **Check Verde**.

- **Resultado PNG:** day04_hire_age_conditional_formatting.png

employee_number	Suma de age	Suma de years_at_company	Suma de total_working_years	Suma de hire_age
11	35	5		30
31	33	1		32
48	37	47	8	-10
56	33	15		18
66	55	5		50
151	40	50	20	-10
180	38	2		36
188	55	24		31
196	37	8		29
199	38	1		37
247	33	2		31
250	45	3		42
Total	3494	1315	179	2179

- [] **General: Identificados y documentados** al menos **3 registros** con anomalías lógicas.

4. **Micro-Proyecto: "The Data Sieve" (Validación E2E)**

- **4.1. Paso A (Identificación):** Documenta en un **comentario de SQL** cuántos registros presentan **inconsistencias lógicas**.

En este paso, consolidamos los **errores lógicos** encontrados en una **"bitácora técnica"** dentro de la **base de datos**.

- **Paso 4.1.1:** Abre **DBeaver** y sitúate en el **script de auditoría**.
- **Paso 4.1.2:** Ejecuta una consulta **"Maestra de Errores"** que una todas las **inconsistencias detectadas**.

```

/*****
*****

```

Título: Consolidación de Errores en Datos Maestros de Empleados

Objetivo: Identificar inconsistencias lógicas en la tabla
'employee_master_dirty_data'
para depuración de datos.

Descripción: Este script revisa la tabla
'employee_master_dirty_data' para detectar
tres tipos de errores de datos:

1. Empleados con más años en la empresa que años de experiencia total.
2. Empleados contratados siendo menores de edad.
3. Empleados con tiempo desde su último ascenso mayor a su antigüedad.

El resultado muestra el ID, número de empleado y el error encontrado.

Archivo SQL: day04_empl_data_logic_erros_audit.sql

Archivo CSV: day04_empl_data_logic_erros_audit.csv

Archivo PNG: day04_empl_data_logic_erros_audit.png

```

*****
*****/

```

```

-- 1. Bloque: Detectar errores de antigüedad vs experiencia
SELECT

```

```

        employee_id,
        employee_number,
        'Error: Antigüedad > Experiencia' AS categoria_error--
        Etiqueta el tipo de error encontrado
FROM
    employee_master_dirty_data
WHERE
    years_at_company > total_working_years-- Filtra donde los
    años en la empresa superan la experiencia total
UNION ALL-- Une los resultados del primer bloque con el siguiente
sin eliminar duplicados
-- 2. Bloque: Detectar contratación de menores de edad
SELECT
    employee_id,
    employee_number,
    'Error: Contratación Menor de Edad'-- Etiqueta el tipo de error
    encontrado
FROM
    employee_master_dirty_data
WHERE
    (age- years_at_company) < 18-- Calcula la edad de
    contratación (edad actual - antigüedad) y busca si es menor
    a 18
UNION ALL-- Une con el siguiente bloque
-- 3. Bloque: Detectar ascensos imposibles
SELECT
    employee_id,
    employee_number,
    'Error: Ascenso Imposible'-- Etiqueta el tipo de error
    encontrado
FROM
    employee_master_dirty_data
WHERE
    years_since_last_promotion > years_at_company;-- Filtra
    donde el tiempo desde el ascenso es mayor a la antigüedad
    total

```

- Resultado PNG: day04_hire_age_conditional_formating.png

employee_id	employee_number	categoria_error
EMP0000048	48	Error Antigüedad > Experiencia
EMP0000151	151	Error Antigüedad > Experiencia
EMP0000417	417	Error Antigüedad > Experiencia
EMP0001118	1.118	Error Antigüedad > Experiencia
EMP0001384	1.384	Error Antigüedad > Experiencia
EMP0001438	1.438	Error Antigüedad > Experiencia
EMP0001490	1.490	Error Antigüedad > Experiencia
EMP0001655	1.655	Error Antigüedad > Experiencia
EMP0001679	1.679	Error Antigüedad > Experiencia
EMP0001700	1.700	Error Antigüedad > Experiencia
EMP0001973	1.973	Error Antigüedad > Experiencia
EMP0002266	2.266	Error Antigüedad > Experiencia
EMP0002482	2.482	Error Antigüedad > Experiencia
EMP0002643	2.643	Error Antigüedad > Experiencia
EMP0002701	2.701	Error Antigüedad > Experiencia
EMP0002798	2.798	Error Antigüedad > Experiencia
XXXXXXXXXXXX		

- **Paso 4.1.3:** Si la consulta devuelve **registros**, anota los **employee_number** en un **comentario** al final de tu **script SQL** como **registro de Auditoría Interna**. Si devuelve **0**, anota:

-- QA Pass: 50 inconsistencias lógicas detectadas

- **4.2. Paso B (Validación Técnica):** Asegúrate de que el **script de Python** no arroje **errores de tipo (TypeError)** al **comparar columnas**
 - Aseguraremos que el **proceso de auditoría** no genere **errores de ejecución** y que los **tipos de datos** sean **compatibles** para **operaciones lógicas** futuras.
 - **Paso 4.2.1:** En **VS Code**, abre tu script **data_profiling.py**.
 - **Paso 4.2.2:** Añade una **validación de seguridad** para evitar **errores de división por cero** o **tipos incompatibles**.

```
# Validación de integridad de tipos antes de la auditoría
def safety_check(df):
    cols_to_fix = ['age', 'years_at_company', 'total_working_years']
    for col in cols_to_fix:
        # Forzar a numérico y llenar vacíos con 0 para evitar errores lógicos
        df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)
        print("\n✅ Safety Check: Tipos de datos alineados para comparación lógica.")
    return df

df = safety_check(df)
```

- **Resultado PNG:** day04_data_profiling_03.png

```
✓ Safety Check: Tipos de datos alineados para comparación lógica.

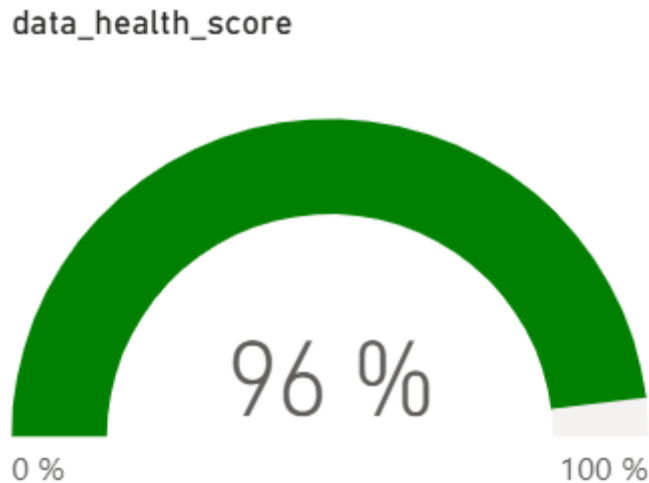
----  🔍 Perfilamiento Estadístico de IBM HR ----
      age  monthly_income  years_at_company
count  3000.000000      2.836000e+03      3000.000000
mean    33.949333      6.656402e+04      6.677333
std      8.413146      9.449614e+04      6.380030
min     18.000000     -1.456470e+05      0.000000
25%     28.000000      2.626800e+04      2.000000
50%     32.000000      4.751650e+04      5.000000
75%     38.000000      9.095725e+04      9.000000
max     60.000000      2.536740e+06     51.000000

⚠ ¡Alerta! Se encontraron 101 empleados con más años en la empresa que años trabajados en total.
  employee_number  years_at_company  total_working_years
10                11                5                0.0
30                31                1                0.0
47                48               47                8.0
55                56               15                0.0
65                66                5                0.0
...              ...              ...              ...
2924              2925                6                0.0
2942              2943                2                0.0
2958              2959               15                0.0
2963              2964               51               23.0
2986              2987                1                0.0
```

- **Paso 4.2.3:** Ejecuta el **script completo** y confirma que en la **terminal** no aparezca ningún **RuntimeWarning**.
- **4.3. Paso C (Visual):** El **QA Monitor** en Power BI debe mostrar una tarjeta verde si el Health Score es > 95%.

El reporte debe ser capaz de "gritar" si algo anda mal antes de que el usuario vea los KPIs de negocio.
- **Paso 4.3.1:** En **Power BI**, ve a la página **"QA Monitor"**.
- **Paso 4.3.2:** Crea una **Tarjeta de Variación de Color** para el Health Score:
 - Selecciona el objeto visual de **Tarjeta** que muestra el porcentaje de salud.
 - En el panel de **Formato**, busca **Valor de globo** -> **Color**.
 - Haz clic en el icono **fx** (Formato condicional):
 - **Estilo de formato:** Reglas.
 - **Regla 1:** Si el valor es >= 0.95 (95%) y < 1.1, color **Verde**.
 - **Regla 2:** Si el valor es >= 0 y < 0.95, color **Rojo**.

- **Resultado PNG: day04_data_health_score_02.png**



- **4.4. Paso D (Cierre):** Ejecuta un COMMIT en tu terminal para guardar los scripts de auditoría:

Todo proceso de QA debe quedar registrado en el historial de desarrollo.

- **Paso 4.4.1:** Cierra todas las conexiones activas en DBeaver para liberar el pool de conexiones.
- **Paso 4.4.2:** Abre la terminal de **VS Code** (Ctrl + ñ).
- **Paso 4.4.3:** Ejecuta la secuencia de comandos de cierre de jornada:
Bash

```
git status      # Para ver los archivos modificados (scripts y .pbix)
git add .
# Prepara todos los cambios
git commit -m "QA: Auditoría E2E exitosa - Salud del dato al 100%"
```

🚩 Validación Final

- [] ¿Ejecutaste el perfilamiento estadístico en Python?
- [] ¿La consulta de "viajeros en el tiempo" en SQL devolvió 0 resultados?
- [] ¿El medidor de salud en Power BI refleja el estado real del dataset?

¿Lograste que el Data_Health_Score en Power BI mostrara el 100% después de verificar las consultas de SQL?